
analytics-service (:4008, optional)

Request flow, file by file

150 questions with answers

12 sections

Event-driven microservices · Go · Kafka · Redis

11 — analytics-service (:4008, optional)

Job: the historical record. Runs only under the `analytics` Docker profile.

```
== Kafka ==
events/consumer.js
  | subscribes to ALL SIX topics
  | fromBeginning: true
  |   ▶ THIS is the payoff of an event log: written LAST, it
  |     replayed 133 events across 4 days – including days
  |     before the service existed. No migration was written.
  |   ▼
events/handlers.js → config/cassandra.js
  |
  | | INSERT donation_events      partition: donationId
  | |                           cluster:  (occurredAt, eventId)
  | |   ▶ eventId is a TIE-BREAK: two events in the same
  | |     millisecond would otherwise overwrite each other
  | |
  | | INSERT events_by_day      partition: day
  | |   ▶ THE SAME EVENTS, STORED TWICE, on purpose.
  | |     donation_events cannot answer "what happened today?"
  | |     without scanning every partition. In Cassandra you
  | |     duplicate data to buy query performance.
  | |
  | | INSERT assignment_outcomes  one row per decision
  | |
  | | UPDATE agent_totals / daily_totals  COUNTER columns
  | |   (!) counters are NOT idempotent – a replay double-counts.
  | |     That is exactly why dedup runs before this, and why
  | |     counters live in their own tables (Cassandra forbids
  | |     counter and non-counter columns in one table).
  | |
  | ▶ Writes go individually, never in a multi-partition BATCH:
  |   that is not an optimisation and not a transaction – it just
  |   makes one coordinator responsible for writes to many nodes.
  |
== HTTP ==
routes/analytics.routes.js → controllers/analytics.controller.js
  ▶ read-only aggregate queries
```

Appendix — the shared skeleton

Repeated here so this page stands alone. Every Node service has these same files doing the same jobs.

Every Node service has these same files doing the same jobs. Once you know this, each service below is just "the skeleton, plus X".

```
src/
├─ index.js           ▶ bootstrap: connect dependencies, THEN listen
├─ app.js             ▶ build the middleware chain (no port binding)
├─ config/
│   ├─ env.js         ▶ read + validate env vars, fail fast at startup
│   ├─ db.js          ▶ mongoose connection
│   └─ redis.js       ▶ redis client (only where Redis is used)
├─ middleware/
│   ├─ requestContext.js ▶ x-trace-id in/out, attaches req.log
│   ├─ requireAuth.js   ▶ verify JWT → req.user; requireRole(...) gate
│   ├─ validate.js      ▶ schema check → 400 with field-level details
│   └─ errorHandler.js ▶ the ONE place every error is formatted
├─ routes/
│   ├─ *.routes.js     ▶ URL → middleware chain → controller
│   └─ *.schemas.js    ▶ the validation schemas
├─ controllers/        ▶ the actual work for one endpoint
├─ models/             ▶ mongoose schemas
└─ utils/
    ├─ jwt.js          ▶ signToken / verifyToken
    ├─ ApiError.js     ▶ typed errors (badRequest, notFound, forbidden...)
    └─ logger.js       ▶ structured JSON logging
```

The universal request pipeline



Why `index.js` and `app.js` are separate. `app.js` builds the Express app but never binds a port; `index.js` connects the database *first*, then listens. That split is what lets the tests drive the app in-process with supertest without binding a port or racing on one — and it means a service never accepts a request it cannot serve.

Why the error handler is last. Every failure, from any middleware or controller, funnels into one place. That is also where donation-service cleans up orphaned uploads — putting cleanup on the happy path missed every request that failed validation.